Traitement de fichiers de données
ÉTUDE DU TRAMWAY LIGNE T3 DE PARIS

1 Mise en situation

1.1 Présentation générale

L'objectif est de développer un programme permettant de traiter des mesures issues d'un accéléromètre de téléphone portable afin d'en extraire des données utiles : position, vitesse, valeurs moyennes...

Les mesures ont été réalisées dans un tramway de la ligne T3a à Paris.



FIGURE 1 – Tramway à la station Cité Universitaire

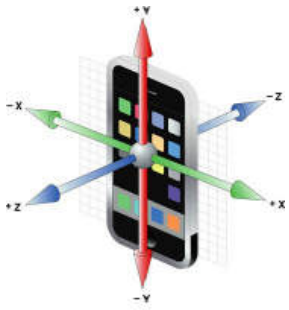
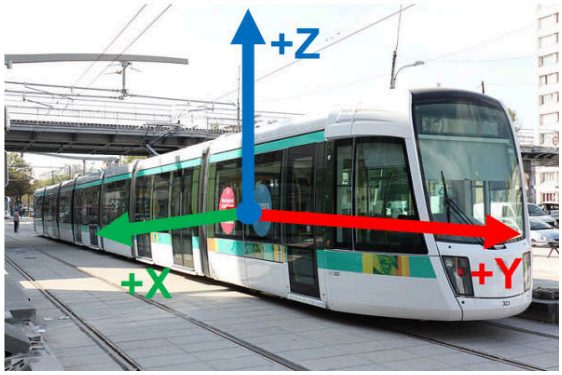


FIGURE 2 – Axes de l'accéléromètre du téléphone portable

1.2 Description de la mesure

Le téléphone portable est tenu par un passager de la façon suivante :

- Le vecteur +X est par conséquent orienté « latéral droit ».
- Le vecteur +Y orienté dans le sens du mouvement ;
- Le vecteur +Z s'oppose à la pesanteur ;



L'essai a été réalisé en ligne droite, aussi on s'intéressera à l'accélération longitudinale (selon Y). Au départ, le tramway est à l'arrêt.

1.3 Problématique

On cherche à déterminer les caractéristiques suivantes :

- Vérifier si le tramway si déplace bien en ligne droite ;
- Courbes de la vitesse et de position ;
- Valeurs moyenne et écart-type sur les moments stationnaires ;
- Valeurs maximales et notamment :
  - ◇ Distance parcourue lors de l'essai ;
  - ◇ Vérifier si la vitesse autorisée n'est pas dépassée (60 km/h en section courante suivant la réglementation du BIRMTG (Bureau Inter-régional des Remontées Mécaniques et des Transports Guidés).

2 Étude de l'accélération

Activité 1

- Ouvrir le fichier `tramway.csv`, il s'agit du fichier de mesures, et observer comment sont organisées les données.

Activité 2

- Créer une fonction `lecture_fichier(nom)` qui permet de lire le fichier `nom` et de renvoyer 2 listes `temps` et `accelerationY` correspondant respectivement aux instants de mesures des accélérations et à l'accélération longitudinale dans l'axe Y.

```
1  def lecture_fichier(nom):
2      # comptage du nombre de lignes
3      fichier = open(nom,'r')   # Ouverture d'un fichier en lecture seule
4      lecture = fichier.readlines()
5      nb_lignes = len(lecture)
6      fichier.close()   # Fermeture du fichier
7
8      # extraction des données utiles
9      fichier = open(nom,'r')
10     fichier.readline()  # saut d'une ligne (non prise en compte des intitulés temps ...)
11
12     # initialisation des listes
13     temps=[]
14     accelerationY=[]
15
16     for i in range(nb_lignes-2):
17         ligne=fichier.readline()            # lecture d'une ligne
18         ligne=ligne.rstrip ("\n\r")         # suppression retour chariot
19         ligne=ligne.replace(",",".")        # changement , en .
20         ligne_data=ligne.split("\t")        # découpage aux tabulations
21
22     # Création des listes
```

```
        temps.append(ligne_data[1])
        accelerationY.append(ligne_data[5])  # extraction accélération longitudinale (selon Y)

        i=i+1   # compteur boucle for
    fichier.close()   # Fermeture du fichier
    return(temps,accelerationY)
```

Activité 3

- Modifier la fonction afin de `lecture_fichier` afin de renvoyer aussi l'accélération latérale (selon X) dans la variable `accelerationX`.

```
accelerationX=[]
temps.append(ligne_data[1])
accelerationY.append(ligne_data[5])  # extraction accélération longitudinale (selon Y)
accelerationX.append(ligne_data[3])  # extraction accélération latérale (selon X)

i=i+1   # compteur boucle for
```

L'unité de l'accélération fournie par l'accéléromètre du téléphone portable est le g ($g = 9{,}81\,\text{m.s}^{-2}$).

Activité 4

- Éxécuter votre fonction sur le fichier `tramway.csv` ;
- Calculer `temps[0] + temps[1]`, `accelerationX[0] + accelerationX[1]`, `accelerationY[0] + accelerationY[1]`. Que se passe t-il ? Quel est le type de `temps[0]` ?
  Le type de `accelerationX[0]`, de `accelerationY[0]` ?
- Créer une fonction `conv2flottant(temps,accelerationX,accelerationY)` qui permet de convertir les 3 listes précédentes en flottant, et faire en sorte que les variables `accelerationX` et `accelerationY` soient exprimées en m.s^{-2}.

```
def conv2flottant(temps,accelerationX,accelerationY):
    # conversion en flottant et en unité SI
    for k in range(len(temps)):
        temps[k]=float(temps[k])
        accelerationX[k]=float(accelerationX[k])*9.81 # m.s-2
        accelerationY[k]=float(accelerationY[k])*9.81
    return(temps,accelerationX,accelerationY)
```

Activité 5

- À l'aide de la bibliothèque `Matplotlib`, écrire une fonction `affiche(L1,L2,T)` qui prend en argument trois listes `L1`, `L2` et `T`, qui permet d'afficher les éléments des 2 listes `L1` et `L2` en fonction de la liste `T` ;

- Afficher les courbes des accélérations longitudinale et latérale en fonction du temps dans une même figure, mais sur 2 tracés distincts (utilisation de `subplot`).

À la lecture des données, on constate qu'il existe une zone où l'accélération longitudinale est quasiment constante.

Activité 6

- Écrire une fonction `moy_ecarts(L,ti,tj)` qui prend en argument une liste `L` et deux temps `ti` et `tj`, et qui retourne la valeur moyenne et l'écart-type de cette liste entre les bornes `temps[i]` et `temps[j]` ;
- Appliquer cette fonction sur le fichier de l'accélération longitudinale.

```
def moy_ecart(L,ti,tj):           # à tester pour ti=8 et tj=12
    pas_temps=temps[1]-temps[0]   # calcul du pas temporel
    # détermination des indices correspondant au temps choisi
    for p in range(len(temps)):
        if temps[p]<ti+pas_temps and temps[p]>ti-pas_temps:
            i=p
        if temps[p]<tj+pas_temps and temps[p]>tj-pas_temps:
            j=p
    som=0
    nb=0
    som_ecart=0
    # calcul de la moyenne
    for k in range(i,j-1):
        som=som+L[k]
        nb=nb+1
    moy=som/nb
    # calcul de l'écart-type
    for k in range(i,j-1):
        som_ecart=(moy-L[k])**2
    from math import sqrt
    ecart=sqrt(som_ecart/nb)
    return(moy,ecart)
```

3 Mise en place d'un filtre numérique

Le signal de l'accélération étant bruité, on souhaite le filtrer à l'aide d'une moyenne glissante. C'est une moyenne qui au lieu d'être calculée sur l'ensemble des n valeurs d'un échantillonnage, est calculée tour à tour sur chaque sous-ensemble de N valeurs consécutives ($N \leq n$). Le sous-ensemble utilisé pour calculer chaque moyenne « glisse » sur l'ensemble des données. On appelle N, l'ordre de la moyenne glissante.

Activité 7

- Écrire une fonction `filtre_mg(L,N)` qui prend en argument une liste `L` ainsi que l'ordre de la moyenne glissante N et qui retourne la liste `L` filtrée ;
- Tester et afficher le résultat obtenu pour différentes valeurs de N.

```
def filtre_mg(L,n):
    mg=[]
    for p in range(len(L)):
        mg_k=0
        for k in range(n):
            mg_k=mg_k+(L[p-k])/n
        mg=mg+[mg_k]
    return mg
```

4 Le conducteur respecte-t-il la vitesse maximale ?

Activité 8

- En utilisant la méthode des rectangles, écrire un programme afin d'obtenir la vitesse du tramway en km/h sous forme d'une liste que l'on nommera `vitesse_rec` ;
- Afficher la courbe de la vitesse en fonction du temps.

```
# détermination de la vitesse par intégration
# méthode des rectangles
pas_temps=temps[1]-temps[0]   # calcul du pas temporel
vitesse_rec=[0] # vitesse initiale
for i in range (len(temps)-1):
    vitesse_rec=vitesse_rec+[vitesse_rec[i]+accelerationY[i]*pas_temps*3.6] # 3.6 : conversion
    en km/h
```

Activité 9

- À l'aide de la fonction `max` native de Python, déterminer la valeur maximale de la vitesse.
- Le conducteur respecte-t-il la réglementation du BIRMTG ?

Activité 10

- La méthode des rectangles pouvant introduire quelques erreurs, modifier le programme afin d'obtenir la liste de vitesse avec la méthode des trapèzes (`vitesse_trap`). Conclure quant à

la différence entre les 2 méthodes.

```
# détermination de la vitesse par intégration
# méthode des trapèzes
pas_temps=temps[1]-temps[0]   # calcul du pas temporel
vitesse_trap=[0] # vitesse initiale
for i in range (len(temps)-1):
    vitesse_trap=vitesse_trap+[vitesse_trap[i]+0.5*(accelerationY[i+1]+accelerationY[i])*
    pas_temps*3.6] # 3.6 : conversion en km/h
```

Activité 11

- Compléter le programme afin d'obtenir la position du tramway (en m) sous forme d'une liste ;
- Afficher la courbe de la position. Quelle est la distance parcourue ?

```
# détermination de la position par intégration
# méthode des rectangles
position=[0] # position initiale
for i in range (len(temps)-1):
    position=position+[position[i]+vitesse_rec[i]*pas_temps/3.6]
```